uc3m | Universidad Carlos III de Madrid

Bachelor in Telematics Engineering

2024-2025

Bachelor Thesis

"Machine Learning-Based Predictive Modeling of Energy Prices"

Rodrigo De Lama Fernández

Emilio Parrado Hernández

Leganés, Spain, June 16th 2025

SUMMARY

This project focuses on the development of a machine learning-based predictive model for electricity prices in Spain. Using historical data from the OMIE (*Operador del Mercado Ibérico de Energía*) and technical analysis indicators, the model aims to accurately forecast hourly energy prices. The study focuses on a single hourly slot, evaluating the performance of various machine learning models, such as Linear Regression, Lasso, and Random Forest. The project focuses on using a sliding window approach alongside feature engineering, employing technical analysis indicators such as moving averages, exponential moving averages, and momentum metrics. The results display the impact of tailoring the features to improve model accuracy and offer insights into the potential of data-driven approaches for energy price forecasting.

Keywords:

Energy, Machine Learning, Sliding Window, Technical Analysis (TA), Technical Indicators

AKNOWLEDGEMENTS

I would like to extend a special thank you to all of the open source contributors of the libraries that have made this project possible.

DEDICATION

Dedicated to my late grandfather, who was not able to see me become an engineer like himself.
To my amazing family.
To my parents, that have always helped me push through hard moments.
To my grandparents who will proudly see me become an engineer.
To my amazing girlfriend that has supported me though all the ups and downs of life.
To my all of friends and colleagues that have accompanied me these years.
To anyone and everyone that has supported me during my years in university.

Here is to the next steps in life.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. INTRODUCTION TO THE PROBLEM

- **1. Explanation of how the price of energy works in Spain**
  - a. Energy
  - b. Prices
- **2. Machine Learning - Lets run through some model overviews, which are available to choose, but without actually referencing anything**

Energy has increasingly become more important with every passing year. Availability is key for a modern high functioning society such as the one we enjoy in Europe, where we are lucky to have a highly advanced and reliable network. Furthermore, electricity is a key resource needed to succeed in the ecological transition that we are currently undergoing world wide, to which electric mobility is a key element of the process. All electric vehicles need convenient and easily available sources of energy to recharge. These can range from electric cars, to buses and trucks, all of which need a to be able to play their role in society.

An important factor in this ever growing necessity is the price of the available energy. But in recent times, that easy and affordable access has changed, and has become more unpredictable. Ever since the commencement of the war between Russia and Ukraine marked and inflection point in the mostly cyclical nature of electricity prices. Prices have gone up considerable in the past two years, and it has been affecting everyone, from big consumers such as companies to individual consumers, having to pay a more expensive electricity bill at the end of the month. This has become a topic of great concern, with society reflecting upon it more often.

This led me to think about pricing, how it was structured, and what to expect as an end consumer. Can we plan our use of energy according to the price? Can the price be predicted accurately? These were some of the questions I had that sparked my interest in looking into energy predictions.

In the initial discussions of this project, we talked about various ways to attack the problem at hand. We thought about a variety of systems but settled on a plan to explore the predictions with the more established methods of Machine Learning. This was done specially since it was a natural continuation of what was learned in the third year subject of Telematics Engineering, Modern Theory of Detection and Estimation.

The first idea we developed was to separate out the project in 24 blocks, one for each hour block to predict. The reason behind splitting the data in 24 blocks was simple, we assumed that data between the same time slot across neighbor days, would be similar, or would follow a certain trend.

Focusing on the innovation side of things, we decided to simplify the problem. Pivoting from creating an algorithm to predict hourly prices, to splitting that into the 24 blocks, and predicting a single one. We decided on the 14^{th} hour of each day, since irrespective of the day of the week, or holiday, it would always be a valley period.

This kind of granular breaking up of data is similar to what a Random Forest does in order to absorb more details and create higher resolution predictions. We made this assumption based on the idea that it would simplify the project substantially.

- In relation to the proposal: Talk about Emilio's background in investment banking at BBVA and his experience using Technical Analysis indicators to aid in model training.

- Talk about the idea of using technical analysis for feature engineering

2. BACKGROUND AND RELATED WORK

How is the price of electricity set in Spain and Portugal?

https://www.omie.es/sites/default/files/inline-files/mercado_diario.pdf

LEER ARTICULO [1] For Iberian electricity market, the OMIE, which stands for *Operador del Mercado Ibérico de Energía*, is the designated operator by the The wholesale price of electricity in Spain and Portugal is set by

https://www.europex.org/members/omie-operador-do-mercado-iberico-de-energia/

What happens when new producers enter the market and sell to the energy pool?

Comment about more typical case studies focusing on energy consumption, since that is something more consistent and predictable

2.1. Training a Non-Stationary Dataset with a Sliding Window Approach

Traditional machine learning models are usually trained on a fix set of data, that is split into various subsets to ensure proper training and evaluation. Starting with the *training set*, which is the largest portion of the dataset and is used to train the model. The algorithm learns patterns, weights, and relationships within the data during this phase. The model's internal parameters are adjusted based on this set.

The *validation set* is used during model development to fine-tune hyperparameters and assess the model's performance on unseen data. It helps detect overfitting, which is when a model performs well on the training data but poorly on new data. The validation set guides model optimization choices without touching the final evaluation set.

The *test set* is strictly separate from the others. It is held out until the end of the training process, to be used to evaluate the final model's generalization capability. It provides an unbiased estimate of how the model will perform on truly unseen data, simulating real-world deployment conditions.

In the context of this project, the dataset is an energy price time series, which is not directly compatible with the standard training methodology, as over time the dataset characteristics might shift, making an arbitrary training set useless. This is because the considered time series is non-stationary.

The proposed solution to mitigate the non-stationary nature of the dataset, is to progressively change our training set. The way of doing this is by creating a *sliding window* of data points that will compose the training set. This sliding window will determine the number of past data points that are relevant for the next prediction, since we will be

effectively transforming the data set into a *stationary* one.

To work with this approach, the training set will be the entirety of the data points contained in the desired sliding window, and the test set will be the following row of data points (price values and features), which will be used to make the next prediction. Essentially, a new model will be created to make the following prediction, being discarded in favor of a new, slightly different model for the next prediction.

2.2. Machine Learning Models Selection

The model selection for the project will consist of three models: the Linear Regressor, the Lasso Regressor, and the Random Forest Regressor.

Linear Regression [2] is one of the most fundamental and widely used statistical techniques for modeling the relationship between a dependent variable and one or more independent variables. The method assumes a linear relationship between input features and the target, expressed as $y = X\beta + \varepsilon$, where β represents the model coefficients and ε is the error term. The model is trained by minimizing the residual sum of squares (RSS), attempting to find the line that best fits the data. Due to its simplicity and interpretability, linear regression will serve as a baseline in many predictive modeling tasks, though it is limited in capturing non-linear relationships and is sensitive to multicollinearity and outliers.

Lasso Regression [3] (Least Absolute Shrinkage and Selection Operator) extends linear regression by introducing L1 regularization, meaning it penalizes the absolute magnitude of the regression coefficients. This penalty term, controlled by a hyperparameter α, encourages sparsity in the model by shrinking some coefficients to exactly zero, effectively performing variable selection. The optimization objective combines the ordinary least squares loss with the L1 penalty:

$$\hat{w} = \arg\min_{w} \left\{ \frac{1}{2n} \|y - Xw\|_2^2 + \alpha \|w\|_1 \right\} \tag{2.1}$$

This characteristic makes Lasso particularly useful in high-dimensional settings where many features may be irrelevant or redundant. While it can reduce overfitting and improve model generalizability, it may also introduce bias by overly shrinking important coefficients, especially when predictors are highly correlated.

Finally, the *Random Forest Regressor* [4] is an ensemble learning method that constructs a collection of decision trees during training, and outputs the average prediction of the individual trees. Each tree in the "forest" is trained on a different sample of the data, and at each split, a random subset of features is considered, which promotes diversity among the trees. This approach helps mitigate the overfitting commonly associated with single decision trees, and improves predictive accuracy and robustness. Unlike linear or Lasso regression, Random Forest does not rely on assumptions of linearity or feature

independence, making it well-suited for capturing complex, non-linear relationships in data. However, this flexibility comes at the cost of reduced interpretability compared to linear models.

2.3. Model Optimization: Hyperparameter Tuning and Feature Engineering

Hyperparameter tuning or *optimization* refers to the process of optimizing the model's training configuration parameters. These are parameters that are not learned from the data during the model training process, they are set before the training process begins and determine how the learning process of the model will develop. For *Lasso* Regression, *alpha* is a hyperparameter that controls the strength of the regularization. For *RandomForest*, `n_estimators` (number of trees) and the `max_depth` (maximum depth of each tree), are a couple of it's hyperparameters. These values for any given model, and the dataset employed in its training, must be experimented in order to find the values that result in the best model performance (e.g., lowest error, highest accuracy) [5].

TODO: Reference academic book: https://www.microsoft.com/en-us/research/wp-content/uploads/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf

Feature Engineering is a crucial step in preparing, and expanding the dataset for training machine learning models. In this project, a particular approach involving *Feature Engineering with Technical Analysis* will be used, creating descriptive features based on the raw price data. Technical analysis, is a methodology for interpreting and analyzing price movements using statistical data and historical patterns to make more accurate predictions [6]. This involves generating technical indicators such as moving averages and indicators for momentum or volatility.

TODO: Reference academic book /article of Technical Analysis

These indicators are not just arbitrary transformations, they are intended to capture the inherent patterns and temporal information implicitly embedded inside the raw price series, theoretically reducing the need for complex time series models. The intention of generating these highly informative features, is to improve the robustness of the machine learning models, as the indicators should encode valuable predictive signals. This allows for a more straightforward model architecture while attempting to leverage the rich temporal dynamics of the data, as proven in other fields such as financial data analysis.

A variety of technical indicators such as *SMA*, *EMA*, *ROC*, *RSI*, *BBWidth* & *ATR* will be used. The *Simple Moving Average* will be used with a variety of window sizes. A shorter SMA could be useful to identify short-term trends in energy prices, capturing recent price movements, while a longer SMA could be useful to capture long-term trends in energy prices, helping identify the overall direction of the market. The *Exponential Moving Average* is similar to a simple moving average, but with the key difference being that it gives more weight to recent prices. This could be valuable to capture more immediate

trends in energy prices. We will use the following *trend-following* indicators:

$SMA_3,\ SMA_5,\ SMA_7,\ SMA_{14},\ SMA_{30},\ SMA_{90},\ SMA_{180}$

$EMA_3,\ EMA_5,\ EMA_7,\ EMA_{14},\ EMA_{30}$

To measure momentum the price *Rate of Change* measures the percentage change between the current price and a price from a previous time period (e.g., 1 day, 7 days ago). It helps identify momentum in the market and is useful to highlight trends or sudden shifts in energy prices. Additionally, the *Relative Strength Index* is another momentum indicator. It is mainly a financial data point that can identify overbought or oversold conditions in the financial markets. Regarding its usage in the energy markets, the intention is it to track whether the price is approaching extreme values, which could indicate a reversal. We will use the following *momentum* indicators:

$ROC_3,\ ROC_5,\ ROC_7,\ ROC_{14},\ ROC_{30}$

$RSI_5,\ RSI_7,\ RSI_{14},\ RSI_{30}$

Finally, to measure volatility we employ indicators such as the Bollinger Band Width and the Standard Deviation. We will use the following *volatility* indicators:

$BBWidth_7,\ BBWidth_{14}$

$STD_7,\ STD_{14},\ STD_{30},\ STD_{90}$

2.4. Evaluation Metrics

To evaluate the predictive performance of the models, a combination of statistical metrics that quantify how closely the predictions align with actual values will be employed. Evaluating using multiple indicators ensures a robust and comprehensive assessment, mitigating the risk of overfitting to a single metric, and providing a deeper understanding of both average-case and worst-case predictive behavior. This multi-metric evaluation framework is essential for iteratively refining the model and selecting the optimal sliding window, and set of hyperparameters.

The *Mean Absolute Error* (MAE) serves as a primary accuracy metric throughout the model development. It calculates the average absolute difference between predicted and actual values, in the same units as the target variable (€/MWh in this context). It offers a simple interpretation of how much, on average, the predictions deviate from reality. Since the MAE treats positive and negative errors equally, it provides a balanced view of model accuracy, which is particularly useful for the hyperparameters tuning process across iterations of the same sliding window. Its simplicity makes it ideal for tracking the performance of each parameter as the and the model training advances along the time series. The formula for the MAE is defined as:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|y_i - \hat{y}_i\right|$$

where y_i is the true value, $\hat{y}_i$ is the predicted value, and n is the total number of observations.

The *Mean Squared Error* (MSE) serves as a second core performance indicator. Unlike the MAE, the MSE penalizes larger errors more heavily, by squaring the differences between predicted and actual values, highlighting outliers or poor predictions. This characteristic makes the MSE particularly effective in identifying models that may perform well on average, but fail under certain conditions. It is especially useful in diagnosing issues related to overfitting or inadequate generalization. The formula for the MSE is defined as follows:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

The *coefficient of determination*, denoted as R^2, provides an intuitive measure of how well the model explains the variance in the target variable. An R^2 value close to 1 suggests that the model captures most of the variability in the data, while a value near 0 indicates a weak correlation between the input features and the target variable. This metric is particularly helpful when comparing multiple models or evaluating the effectiveness of adding additional data points with feature engineering, as it gives a high-level summary of the overall model fit. The coefficient of determination (R^2) may be calculated with the following formula:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

where $\bar{y}$ is the mean of the observed values.

Beyond average errors, the prediction accuracy can be also evaluated in terms of *percentiles* to understand how well the model performs across different segments of the distribution. For instance, checking the 70th, 80th, and 90th percentiles of error allows for an assessment of both typical and extreme prediction behaviors. This adds granularity to the evaluation process, highlighting whether the model is consistently accurate or only performs well under specific conditions. The percentiles will be calculated as the top performers in a given range, meaning only the example, 70%, 80% or 90% of most accurate predictions would be considered.

Pivoting from the best percentiles, to the worst, the concept of *expectation shortfall*, commonly used in financial risk analysis, was included as a measure of tail risk in model predictions. This metric calculates the average prediction error among the worst-performing cases (e.g., the top 5% of errors), providing a quantitative estimate of the magnitude of extreme failures. In the context of energy prices, it helps us quantify how bad the prediction is when it fails, specially with over-estimations by hundreds of euros per MWh, since under-estimations shall be treated as 0, as we will work with the assumption that the price is not typically negative. For this application, the expectation shortfall may be upper bounded with a domain-specific limit (e.g. 1000 €/MWh) to avoid distortion from implausible outlier predictions. This measure adds a risk-aware perspective to the model evaluation, ensuring that occasional large errors are accounted.

2.5. Related Work

Other ways of predicting energy prices instead of Machine Learning algorithms

LTSMs - Temporal Series?

Energy consumption predictions?

3. PROPOSAL: SYSTEM DESCRIPTION

This chapter outlines the design of the system developed for the project, describing the complete *data processing pipeline* that was constructed. This will be explained starting from a blank state, detailing the complete set-up process required to run the project. Following this initial configuration, the data preparation stage will be outlined. This will be continued by the feature matrix creation process, concluding with the training of the machine learning models and the chosen evaluation method.

The following block diagram models the design of the system pipeline design in its most basic form:

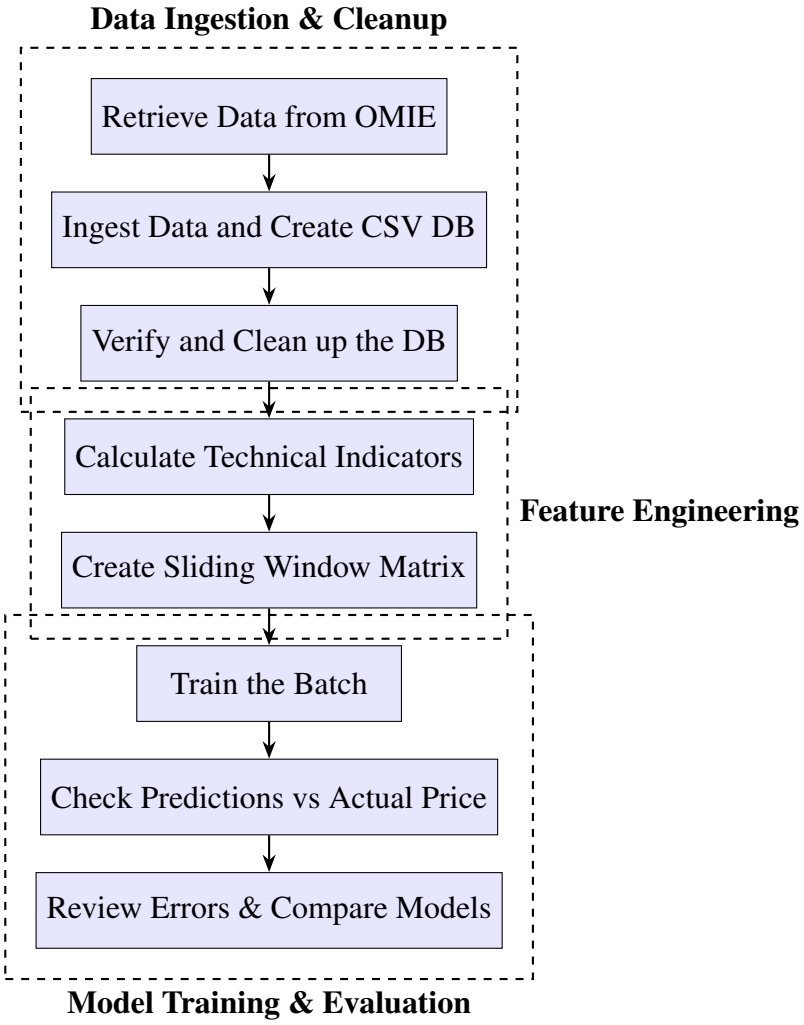

Fig. 3.1. Theoretical Block Diagram of the Project Pipeline

3.1. Prior Set Up and Configuration

Before getting into the system explanation, there are some important pre-requisites that must be met in order to run the project. The project was developed and tested with Python 3.13 [7], so for any recreation of the project, it is highly recommendable to use the same version. For additional assurance of reproducibility, and as a universal best practice for Python development, it is strongly advisable to set up a new virtual environment [8]. Alternatively, another system that would achieve the same goal of reproducibility, and dependency conflict avoidance would be the use of Miniconda [9]. In this project, the built in Python module *venv* was chosen, with which a virtual environment can be easily created and configured with the project dependencies using the following commands.

Create the virtual environment:

```
python -m venv .venv
```

Activating the new virtual environment:

```
in macOS
source .venv/bin/activate

in Windows
.venv\Scripts\Activate.ps1
```

To reproduce the exact environment used in the project, a *requirements.txt* file must be created with the following contents:

```
ipykernel==6.29.5
matplotlib==3.10.1
numpy==2.2.3
pandas==2.2.3
requests==2.32.3
scikit-learn==1.6.1
ta==0.11.0
```

To install the dependencies run:

```
pip install -r requirements.txt
```

These libraries were specifically selected to aid in the development of the project. Most of the project was written in Jupyter Notebooks, which are dependent on the *ipykernel* package. To aid in the retrieval of the data, the *requests* package was used to execute HTTP requests. For the manipulation of the data points, *pandas* was used to house the

data structures, *numpy* for mathematical operations using arrays, and the *ta* technical analysis library was used to compute the technical indicators. Finally the machine learning algorithms were provided by the *scikit-learn* library.

Following the completion of this prior set up required for the project, the next sections will detail in depth the different steps involved in the system pipeline.

3.2. Data Preparation: Ingestion and Cleanup

The first step in any data preparation process would be to retrieve the raw data from the original source, in this case this means downloading it from the OMIE. It is publicly available data easily discoverable through their webpage [10]. The data collected was all that was initially available towards the end of 2024, with some extraordinary retrievals done later on in 2025 to further complete the set with more data points.

In this scenario, it meant retrieving data from 2018 onward. While for the 5 first years available the data was self contained in a compressed archive file, the rest from 2023 to our current 2025 was not. This meant that to retrieve the files, manually downloading each resource was required. In order to streamline this process, a short Python script was created to build the necessary URIs to retrieve the files with an HTTP GET request.

The following Python script was written with modularity in mind for ease of use. It firstly creates the filenames according to the OMIE's file naming convention, inserting them into a list. This list is essential to successfully build the URIs with which the HTTP GET request will be made. Finally the function `download_files(urls)` makes the request, retrieving the files.

```
import requests
from datetime import datetime, timedelta

# Usage instructions:
    # Set file name to save the filenames to download
    # Set start and end dates

file_path = 'to_download.txt'
start_date = datetime(2023, 1, 1)
end_date = datetime(2025, 3, 18)

# Compose filenames to download
def compose_filenames():
    start_date = start_date
    end_date = end_date

    # Generate the entries for the number of days comprehended
    to_generate = (end_date - start_date).days + 1
    entries = []
    for i in range(0, to_generate):  # x days from start to end
```

```
        date_entry = start_date + timedelta(days=i)
        entries.append(f'marginalpdbc_{date_entry.strftime("%Y%m%d")}.1')

    # Save to a .txt file
    with open(file_path, 'w') as file:
        for entry in entries:
            file.write(entry + '\n')

# Example URI to build
# https://www.omie.es/es/file-download?parents%5B0%5D=marginalpdbc&filename=marginalpdbc_20230102.1
def read_filenames_and_compose_urls(file_path):
    base_url = "https://www.omie.es/es/file-download?parents%5B0%5D=marginalpdbc&filename="

    # Read the filenames from the .txt file
    with open(file_path, 'r') as file:
        filenames = [line.strip() for line in file if line.strip()]

    # Compose the URLs
    urls = [base_url + filename for filename in filenames]

    return urls

def download_files(urls):
    for url in urls:
        # Extract the filename from the URL
        filename = url.split('=')[-1]

        # Send a GET request to download the file
        response = requests.get(url)

        # Check if the request was successful and write to file
        if response.status_code == 200:
            # Write the content to a file with the extracted filename
            with open(filename, 'wb') as file:
                file.write(response.content)
            print(f"Downloaded {filename}")
        else:
            print(f"Failed to download {filename}")

# Execution
compose_filenames()
urls = read_filenames_and_compose_urls(file_path)
download_files(urls)
```

The newly downloaded raw data was manually organized into its corresponding folder per year. An example filename is the following: `marginalpdbc_20230102.1`. This is an important

organizational step for the ingestion of the data, as the next procedure involves a script that iterates through these folders, identifying all files that start with `marginalpdbc_`, and parsing them into an easier to use form, a Pandas DataFrame [11].

When a valid file is identified while iterating through the *year* folders, the script reads its content. It will perform some cleanup steps, such as removing the first and last rows which are a standard header and footer across all files. It then parses the remaining data, which is separated by semicolons, into a Pandas DataFrame for that specific day. It will name the columns to *Year*, *Month*, *Day*, *Hour*, *MarginalPT* (Portuguese Energy Price), and *MarginalES* (Spanish Energy Price). It will also filter out any rows where the *Year* column is not a digit, ensuring data integrity. The columns are then converted to their appropriate data types, *integers* for date/time components and floats for marginal prices. Each processed daily dataset is then temporarily stored in a list.

After processing all the individual files, the script concatenates all the daily data from the list of DataFrames into a single, comprehensive DataFrame. It will then construct a *Datetime* column built from the *Year*, *Month*, *Day*, and *Hour* columns. It will also handle some peculiar cases where an *Hour* value of 25 might appear (likely due to daylight savings time zone adjustments) by converting it to 0. This *Datetime* column is then set as the DataFrame's index. Finally, it removes the individual date and hour columns as well as the *MarginalPT* column, leaving only the *MarginalES* as the primary target variable, along with the *Datetime* index.

This clean and combined dataset will be saved as a CSV file for ease of use, `raw_data.csv`. The script then performs further data cleanup by reloading the `raw_data.csv`, and sorting it by *Datetime*, saving it as `processed_data.csv`. The whole process concludes by verifying the completeness of the hourly timestamps within the processed data, generating the full range of hourly timestamps that should be there, and checking for any discrepancies. If no missing timestamps are found, it confirms successful data processing; otherwise, it reports the missing timestamps, indicating an error in the data collection or processing.

For the previous in-depth explanation, you may find the of Python script that was created for the desired data ingestion and cleanup tasks:

```
import pandas as pd
import os

# Base path to the folder containing the year by year data folders
base_path = '../TFG/data/'

# Initialize an empty list to store DataFrames
all_data = []

# Iterate through each year folder and process all files
for root, dirs, files in os.walk(base_path):
    for file in files:
        # Check if the file starts with 'marginalpdbc_' (ignoring
           the rest, including the extension)
        if file.startswith('marginalpdbc_'):

            # Read the file
```

```
17                file_path = os.path.join(root, file)
18                with open(file_path, 'r', encoding='utf-8') as f:
19                    lines = f.readlines()
20
21                # Remove the first line (MARGINALPDBC;) and the last
                      line (*)
22                data_lines = lines[1:-1]
23
24                # Convert the list of semicolon-separated strings into a
                       DataFrame
25                daily_data = pd.DataFrame([line.strip().split(';') for
                      line in data_lines])
26
27                # Remove trailing ';' from the last column
28                if daily_data.shape[1] > 6:
29                    # Drop empty columns
30                    daily_data = daily_data.iloc[:, :6]
31
32                # Assign column names
33                daily_data.columns = ['Year', 'Month', 'Day', 'Hour', '
                      MarginalPT', 'MarginalES']
34
35                # Remove rows with invalid data (in case accidental
                      empty rows, or non-numeric values, get included)
36                daily_data = daily_data[daily_data['Year'].str.isdigit()
                      ]  # Only keep rows where 'Year' is a number
37
38                # Convert the columns to the appropriate data types
39                daily_data = daily_data.astype({
40                    'Year': int, 'Month': int, 'Day': int, 'Hour': int,
                          'MarginalPT': float, 'MarginalES': float
41                    })
42
43                # Append daily data to the list
44                all_data.append(daily_data)
45
46    # Concatenate all daily DataFrames into one big DataFrame
47    full_data = pd.concat(all_data, ignore_index=True)
48
49    # Create a datetime column from Year, Month, Day, and Hour
50    # Adjust the 'Hour' column to deal with the potential 25th hour
          issue in case of timezone adjustment
51    full_data['Hour'] = full_data['Hour'].apply(lambda x: 0 if x == 25
          else x)
52    full_data['Datetime'] = pd.to_datetime(full_data[['Year', 'Month', '
          Day', 'Hour']], errors='coerce')
53
54    # Set the 'Datetime' column as the index
55    full_data.set_index('Datetime', inplace=True)
56
57    # Drop the now unnecessary time columns and 'MarginalPT', as we are
```

```
    focusing on the Spanish market
full_data.drop(['Year', 'Month', 'Day', 'Hour', 'MarginalPT'], axis
    =1, inplace=True)

# Save the full dataset to a CSV file
full_data.to_csv('../../data/raw_data.csv')

# Post-ingest database cleanup:
# Load and sort the data by the 'Datetime', and save the sorted data
     to a new file
df = pd.read_csv('../../data/raw_data.csv')
df = df.sort_values(by='Datetime')
df.to_csv('../../data/processed_data.csv', index=False)

# Read the CSV file
df = pd.read_csv('../../data/processed_data.csv')
df['Datetime'] = pd.to_datetime(df['Datetime']) # Parse 'Datetime'
    column to datetime objects

# Generate the complete range of hourly timestamps
full_range = pd.date_range(start=df['Datetime'].min(), end=df['
    Datetime'].max(), freq='h')

# Find any missing timestamp
missing_timestamps = full_range.difference(df['Datetime'])

# If successful, print a message
if missing_timestamps.empty:
    print("Data ingestion completed successfully.")

# Else, print an error message
else:
    print("Error: Missing timestamps found. Check the data.")
    print("Missing timestamps:\n missing_timestamps")

    print("Data ingestion completed with errors.")
```

After completing this ingestion and cleanup process, the dataset will be reduced to just the 14^{th} hour data point. The following code will create a the final database file, which will house the singular *Datetime* value, and the *MarginalES* data point:

```
import pandas as pd

# Load the original dataset
data_path = '../../data/processed_data.csv'
df = pd.read_csv(data_path, parse_dates=['Datetime'])

# Define the hour to predict and filter down to the 14:00H data
    point
hour_to_predict = 14
```

```
df_hour = df[df['Datetime'].dt.hour == hour_to_predict].copy()

# Save the final database
df_hour.to_csv("../../data/hour_14_metrics.csv", index=False)
```

To aid in the predictions, the dataset is enhanced with more features. We can calculate a variety of different metrics to aid the training of the model. These can range from simple metrics such as the day of the week, the month of the year, or if that day is a weekend or not, to more complex, such as technical indicators. By adding these auxiliary points of data derived from *technical analysis*, we can further expand the context size that our models will be able to learn from, ideally improving their prediction accuracy capabilities.

To generate these additional features, and the strategically chosen technical indicators, the main database `processed_data.csv` is read into a DataFrame that will be progressively expanded with new feature columns as the respective metrics are calculated. The resulting DataFrame will be saved to a secondary database `ta_metrics_hour_14.csv`. The following code is an example of the process that was employed to calculate the different indicators, using the open source Python library *ta* [12]:

```
import pandas as pd
import ta  # bukosabino's Technical Analysis Library
import numpy as np

# Load the data
data_path = 'data/processed_data.csv'
df = pd.read_csv(data_path, parse_dates=['Datetime'])

# Reduce dataset to only to the 14th hour
hour_to_predict = 14
df = df[df['Datetime'].dt.hour == hour_to_predict].copy()

# The "window" parameter is measured in days

# Trend-following - SMA and EMA
# Simple Moving Average
df[f'SMA_{3}'] = ta.trend.SMAIndicator(close=df['MarginalES'],
    window=3).sma_indicator()
df[f'SMA_{5}'] = ta.trend.SMAIndicator(close=df['MarginalES'],
    window=5).sma_indicator()
df[f'SMA_{7}'] = ta.trend.SMAIndicator(close=df['MarginalES'],
    window=7).sma_indicator() # past week trend
df[f'SMA_{14}'] = ta.trend.SMAIndicator(close=df['MarginalES'],
    window=14).sma_indicator()
df[f'SMA_{30}'] = ta.trend.SMAIndicator(close=df['MarginalES'],
    window=30).sma_indicator() # past month trend
df[f'SMA_{90}'] = ta.trend.SMAIndicator(close=df['MarginalES'],
    window=90).sma_indicator()
df[f'SMA_{180}'] = ta.trend.SMAIndicator(close=df['MarginalES'],
    window=180).sma_indicator() # past half year trend
```

```
24
25  # Exponential Moving Average
26  df[f'EMA_{3}'] = ta.trend.EMAIndicator(close=df['MarginalES'],
        window=3).ema_indicator()
27  df[f'EMA_{5}'] = ta.trend.EMAIndicator(close=df['MarginalES'],
        window=5).ema_indicator()
28  df[f'EMA_{7}'] = ta.trend.EMAIndicator(close=df['MarginalES'],
        window=7).ema_indicator()
29  df[f'EMA_{14}'] = ta.trend.EMAIndicator(close=df['MarginalES'],
        window=14).ema_indicator()
30  df[f'EMA_{30}'] = ta.trend.EMAIndicator(close=df['MarginalES'],
        window=30).ema_indicator()
31
32  # Momentum - Rate of Change (ROC) and RSI
33  # Rate of Change
34  df[f'ROC_{3}'] = ta.momentum.ROCIndicator(close=df['MarginalES'],
        window=3).roc()
35  df[f'ROC_{5}'] = ta.momentum.ROCIndicator(close=df['MarginalES'],
        window=5).roc()
36  df[f'ROC_{7}'] = ta.momentum.ROCIndicator(close=df['MarginalES'],
        window=7).roc()
37  df[f'ROC_{14}'] = ta.momentum.ROCIndicator(close=df['MarginalES'],
        window=14).roc()
38  df[f'ROC_{30}'] = ta.momentum.ROCIndicator(close=df['MarginalES'],
        window=30).roc()
39
40  # Relative Strength Index
41  df[f'RSI_{5}'] = ta.momentum.RSIIndicator(close=df['MarginalES'],
        window=5).rsi() # rapid momentum changes
42  df[f'RSI_{7}'] = ta.momentum.RSIIndicator(close=df['MarginalES'],
        window=7).rsi()
43  df[f'RSI_{14}'] = ta.momentum.RSIIndicator(close=df['MarginalES'],
        window=14).rsi() # biweekly momentum
44  df[f'RSI_{30}'] = ta.momentum.RSIIndicator(close=df['MarginalES'],
        window=30).rsi()
45
46  # Volatility - Bollinger Bands Width and Standard Deviation
47  # BB Width (Bollinger Bands Width)
48  df['BB_Width_7'] = ta.volatility.BollingerBands(close=df['MarginalES
        '], window=7, window_dev=2).bollinger_wband()
49  df['BB_Width_14'] = ta.volatility.BollingerBands(close=df['
        MarginalES'], window=14, window_dev=2).bollinger_wband()
50
51  # Standard Deviation (STD)
52  df['STD_7'] = df['MarginalES'].rolling(window=7).std()
53  df['STD_14'] = df['MarginalES'].rolling(window=14).std()
54  df['STD_30'] = df['MarginalES'].rolling(window=30).std()
55  df['STD_90'] = df['MarginalES'].rolling(window=90).std()
56
57  # Additional contextual features
58  df['month'] = df['Datetime'].dt.month
```

```
df['day_of_week'] = df['Datetime'].dt.dayofweek  # Monday=0
df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)

# Drop missing values generated by rolling calculations
df.dropna(inplace=True)

# Check for NaN or infinity values (specially in ROC)
print("NaN values per column:\n", df.isna().sum())
print("Infinity values per column:\n", df.isin([np.inf, -np.inf]).
    sum())

# Drop NaN or infinity values and interpolate
df = df.replace([np.inf, -np.inf], np.nan).dropna()
df = df.interpolate()

# Save the dataset with metrics
df.to_csv(f'data/ta_metrics/final_price_ta_metrics.csv', index=False
    )
```

3.3. Feature Matrix Creation

The creation of a *feature matrix* is a key step in the process of training a supervised learning model. In this context, the objective is to structure historical energy price data into a form that can be used effectively to predict future prices. This transformation of the raw data is critical when dealing with time-series data using traditional machine learning models, since observations must be ordered correctly in time as they often exhibit autocorrelation.

The intent of building the feature matrix consists on creating one unique matrix that can be used across any desired model. The feature matrix denoted as X, represents the input data used to train the model, while the target / label vector y contains the values the model is intended to learn how to predict.

For a given time series T:

$$T = \begin{Bmatrix} t_1 \\ t_2 \\ t_3 \\ t_4 \\ t_5 \\ t_6 \end{Bmatrix} = \begin{Bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{Bmatrix}$$

Its simple feature matrix can be represented as:

$$X = \begin{Bmatrix} x_1 & x_2 & x_3 \\ x_2 & x_3 & x_4 \\ x_3 & x_4 & x_5 \end{Bmatrix}, \quad y = \begin{Bmatrix} y_1 \\ y_2 \\ y_3 \end{Bmatrix}$$

Where each row of X corresponds to a window of past values, and its corresponding label y is

the value that follows that window, which would be the actual energy price for the current day:

$$y_1 = x_4$$
$$y_2 = x_5$$
$$y_3 = x_6$$

This setup reflects the temporal dependency in the time-series data, where future values are predicted based on a certain number of past observations. This approach allows the machine learning models to "learn" these implicit patterns in the considered sliding window, enabling them to better predict the next value.

Incorporating technical analysis indicators into the feature matrix introduces additional information such as moving averages or the rate of change calculated over historical price windows, These enrich the model's inputs and are added as extra columns to the feature matrix:

$$X = \begin{Bmatrix} x_1 & x_2 & x_3 & ta_{11} & ta_{21} & \cdots & ta_{n1} \\ x_2 & x_3 & x_4 & ta_{12} & ta_{22} & \cdots & ta_{n2} \\ x_3 & x_4 & x_5 & ta_{13} & ta_{23} & \cdots & ta_{n3} \end{Bmatrix}$$

Where:

- x_i are raw price values from previous time steps,
- n is the number of indicators,
- ta_{ji} is the value of the j-th technical indicator computed based on the last price input of row i, i.e., x_{i+2} in this 3 previous days example.

The target / label vector will remain unchanged:

$$\mathbf{y} = \begin{Bmatrix} y_1 \\ y_2 \\ y_3 \end{Bmatrix}$$

This design is motivated by the nature of the energy market data, which is *non-stationary*, as the statistical properties of electricity prices (e.g. mean, variance) change over time due to evolving supply and demand dynamics, regulatory interventions, and renewable energy contributions. Since the dataset has inherent *temporal dependencies*, as prices are typically dependent on recent values, making the assumption of independent and identically distributed (i.i.d.) samples would be invalid. The aim is to identify *local trends*, implementing a sliding windows approach allows the model to capture short-term patterns and trends without requiring the data to be stationary.

By using a sliding window mechanism, the models will be trained on a continuous stream of overlapping data segments. This not only increases the number of training samples, enhancing generalization, but also aligning with the real-world scenario of forecasting the next value based on a recent history of observations. The ultimate aim is to build feature matrix using the sliding window approach to provide a robust and flexible representation of temporal data, allowing traditional machine learning models to operate more effectively in a domain typically reserved for time-series-specific methods.

The following Python Script consists on the code necessary to create the simple feature matrix, which was initially generated based off *MarginalES* values, with the desired sliding window width for that training batch:

```
import pandas as pd
import numpy as np

def create_feature_matrix(data, price_feature_window):
    """
    Creates a feature matrix where each row is a sliding window of prices,
    and a corresponding target vector containing the next price value.

    Parameters:
    ----------
    - data: DataFrame with 'Datetime' and 'MarginalES' columns
    - price_feature_window: Size of the prices sliding window

    Returns:
    -------
    - X: DataFrame with sliding windows as rows
    - y: Series with target values (next price after each window)
    """
    # Extract the MarginalES column
    if 'MarginalES' in data.columns:
        prices = data['MarginalES'].values
    else:
        # Assume it's the second column (index 1)
        prices = data.iloc[:, 1].values

    # Create empty matrices
    X = np.zeros((len(prices) - price_feature_window, price_feature_window))
    y = np.zeros(len(prices) - price_feature_window)

    # Fill the matrices with the sliding windows and targets
    for i in range(len(prices) - price_feature_window):
        X[i, :] = prices[i:i+price_feature_window]  # Window of prices
        y[i] = prices[i+price_feature_window]       # Next price after window

    # Convert to DataFrame/Series for easier use in training
    return pd.DataFrame(X), pd.Series(y)
```

For an example run we can retrieve from the database a sliding window of 6 days:

```
              Datetime  MarginalES
187 2019-01-01 14:00:00       65.88
```

```
188 2019-01-02 14:00:00       63.16
189 2019-01-03 14:00:00       66.70
190 2019-01-04 14:00:00       69.17
191 2019-01-05 14:00:00       64.00
192 2019-01-06 14:00:00       64.86
```

```
import pandas as pd
from utils.matrix_builder import create_feature_matrix

# read data
csv_hour_file = '../data/ta_metrics/final_price_ta_metrics.csv'
simple_df = pd.read_csv(csv_hour_file, parse_dates=['Datetime'])
simple_df = simple_df[['Datetime', 'MarginalES']]

# Sliding window of 6 days
train_start_date = '2019-01-01'
train_end_date = '2019-01-07'

simple_subset = simple_df[(simple_df['Datetime'] >= train_start_date
    ) & (simple_df['Datetime'] <= train_end_date)]

# Number of previous days as columns (features)
price_feature_window = 3

X_simple, y_simple = create_feature_matrix(simple_subset,
    price_feature_window)
```

We may confirm the simple matrix is correctly built with the following results:

```
print(X_simple)
0      1      2
0  65.88  63.16  66.70
1  63.16  66.70  69.17
2  66.70  69.17  64.00

print(y_simple)
0    69.17
1    64.00
2    64.86
```

This function was later modified and extended to incorporate additional feature columns such as the focus of our proposal, the technical indicators. These are the previously calculated metrics in order to expand the context our models will be able to train with. The following code displays the flexible and auto-adaptable code for any number of inputted metrics (columns) obtained from the secondary feature enhanced database, which includes the additional technical indicators:

```
def create_ta_feature_matrix(dataframe, price_feature_window):
```

```
    """
    Creates a sliding window dataset for time series forecasting where each row contains:
    1. A window of historical prices (right-aligned)
    2. Feature values from the most recent point in the window
    3. Target value (next price after the window)

    Parameters:
    ----------
    dataframe : pandas.DataFrame
        DataFrame containing at minimum 'Datetime' and 'MarginalES' columns,
        plus any additional feature columns
    price_feature_window : int
        Number of data points to include in each window of prices

    Returns:
    -------
    X : pandas.DataFrame
        Features DataFrame with:
        - Historical prices labeled as 'price_t-n' through 'price_t-1'
        - All additional features from the original dataframe
    y : pandas.Series
        Target values (price at time t)
    """

    # Input validation
    if price_feature_window < 1:
        raise ValueError("Feature window size must be at least 1")
    if len(dataframe) <= price_feature_window:
        raise ValueError(f"DataFrame must have more rows ({len(dataframe)}) than price_feature_window ({price_feature_window})")
    if 'Datetime' not in dataframe.columns or 'MarginalES' not in dataframe.columns:
        raise ValueError("DataFrame must contain 'Datetime' and 'MarginalES' columns")

    X, y = [], []

    # Extract price data and features
    df_prices = dataframe[['Datetime', 'MarginalES']]
    df_features = dataframe.iloc[:, 2:] # Exclude 'Datetime' and 'MarginalES'
    feature_names = df_features.columns.tolist()

    # Create samples from the data
    for i in range(price_feature_window, len(df_prices)):
        # Extract the window for prices as features (right-aligned)
        window = df_prices.iloc[i-price_feature_window:i, 1].values.
```

```
            flatten()

        # Extract corresponding feature row (from the most recent
            point in the window)
        feature_row = df_features.iloc[i-1].values.flatten()

        # Concatenate window prices with feature row
        X.append(np.concatenate((window, feature_row)))
        y.append(df_prices.iloc[i, 1])  # Predict current price

    # Create column names for the prices feature window
    price_columns = [f'price_t-{price_feature_window-i}' for i in
        range(price_feature_window)]

    # Return DataFrame and Series with proper column names
    X_df = pd.DataFrame(X, columns=price_columns + feature_names)
    y_series = pd.Series(y, name='price_t')
    return X_df, y_series
```

Furthermore, for an example run of the matrix creation with the technical indicators we can retrieve the extended database and select the same date range as in the previous example for comparison:

```
import pandas as pd
from utils.matrix_builder import create_feature_matrix

# read data
csv_hour_file = '../data/ta_metrics/final_price_ta_metrics.csv'
features_df = pd.read_csv(csv_hour_file, parse_dates=['Datetime'])

# Sliding window of 6 days
train_start_date = '2019-01-01'
train_end_date = '2019-01-07'

feature_subset = features_df[(features_df['Datetime'] >=
    train_start_date) & (features_df['Datetime'] <= train_end_date)]

# Number of previous days as columns (features)
price_feature_window = 3

X_extended, y_extended = create_expanded_feature_matrix(
    feature_subset, price_feature_window)
```

Here we may observe as similar matrix ton the earlier one, expanded laterally to include the new feature columns, such as the technical indicators and additional *"Datetime"* related context:

```
print(X_extended)
   price_t-3  price_t-2  price_t-1      SMA_3   SMA_5      SMA_7     SMA_14  \
0      65.88      63.16      66.70  65.246667  64.974  63.842857  64.975000
```

```
1      63.16      66.70      69.17  66.343333  66.026  64.490000  65.401429
2      66.70      69.17      64.00  66.623333  65.782  65.434286  65.330000

      SMA_30     SMA_90    SMA_180  ...     RSI_30  BB_Width_7  BB_Width_14  \
0  64.886000  65.044333  67.227333  ...  50.823486   17.871701    15.638302
1  64.988333  65.038667  67.272889  ...  52.153121   21.198596    16.528083
2  64.947333  65.184222  67.267611  ...  49.268675   11.634263    16.685668

      STD_7    STD_14    STD_30    STD_90  month  day_of_week  is_weekend
0  3.080999  2.636139  2.620006  4.626188    1.0          3.0         0.0
1  3.691585  2.804414  2.726835  4.620755    1.0          4.0         0.0
2  2.055690  2.828060  2.732317  4.369895    1.0          5.0         1.0

print(y_extended)
0    69.17
1    64.00
2    64.86
```

3.4. Machine Learning Model Training

There are multiple strategies that can be implemented for an effective model training, depending on if the dataset is *stationary* or not. An *incremental* approach can be taken when de data is *stationary*, since in this case, the accumulating the data points will benefit the model's training. An *adaptive* approach is in order when the dataset is of *non-stationary* nature. Similar to an exponential moving average, where the newer data holds more weight, while the older data's importance is progressively minimized, this investigation's models were trained with a sliding window approach, where the older data points are cut off. This training method "forgets" the earliest values as they are no longer relevant in order to make the latest prediction, which is is the case in the energy prices database

The idea behind the previously introduced *adaptive learning* approach is simple, we will create a 1 day step loop across the complete time series. A finite set of days will be used in training each model, with each instance being utilized for one single prediction. The training phase will consist on searching the optimal sliding window of days, with the calculated set of features. In each iteration, the model will be trained with a set feature matrix, and will then be fed with the next feature row to test its prediction abilities.

Continuing with our previous example, the model will be trained with the feature matrix X.

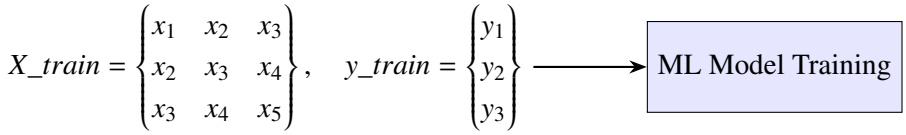

Fig. 3.2. Simple Training Example

In order to predict a new value, for example, $\hat{y_4}$, the model will be fed the next row of feature values $\{x_4, x_5, x_6\}$, where x_6 will be our last known value, y_3.

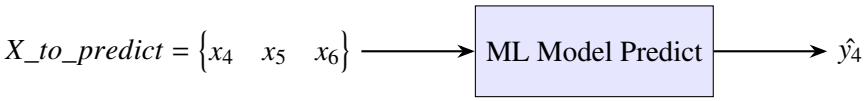

Fig. 3.3. Simple Prediction Example

TODO: Review models

The model selection for the project will consist of three models: the Linear Regressor, the Lasso Regressor, and the Random Forest Regressor. The first of the models, the *Linear Regressor* will serve as our baseline. This model assumes that the target variable can be approximated as a linear combination of the input features.

Our second model, the *Lasso Regressor*, extends the linear model by introducing L1 regularization, which has the ability to penalize unused features. This penalty encourages sparsity in the feature weights, effectively performing feature selection. By the selection of its hyperparameter *alpha* (α), we can modify the strength of the models regularization, potentially improving the model's performance. By tuning *alpha*, we can determines how strongly the model penalizes non-zero coefficients, potentially improving generalization and reducing overfitting. To optimize our predictions, we will later experiment across a variety of *alpha* hyperparameter values.

The third and final model considered in this investigation, is the *Random Forest Regressor*, an ensemble learning method based on decision trees. It constructs multiple decision trees during training, and aggregates their outputs by averaging the regression tasks. We can tune the training process by adjusting parameters such as `n_estimators` (number of trees) and the `max_depth` (maximum depth of each tree). The average output can be a potentially limiting factor for its predictions, since even if it is more capable to learn more complex patterns than the aforementioned models, it will never be able to predict any peaks. This model has been included as a non-linear alternative to validate the performance of the linear models and capture more complex relationships within the data.

END TODO

Regardless of the chosen model, the training strategy will be as follows: After importing the complete dataset, we will define the number of days you want to include in the sliding window. This determines how many days from the time series will be included in the training. At the same point, we will select the desired number of previous days to be included in the feature matrix as columns. Depending on the selection of the sliding window size, there will be more or less models to train.

After initializing the variables where the predictions, actual values, and their time stamps will be stored at, the training loop will begin. Here, a new model will be trained per iteration as seen in Figure 3.2, and a new value will be predicted as seen in Figure 3.3. Once all models and predictions have been processed, the `prediction_df` DataFrame will be available for evaluation of the predictive precision of the training loop.

To finalize the theoretical system description, the following example script demonstrates in

Python code the basic concept of our sliding window training process, in which any given model could be used. However, it is a basic example that does not consider any hyperparameter tuning. This will be covered in the following section, the dedicated *Experimentation* chapter.

```
import pandas as pd
from utils.matrix_builder import create_expanded_feature_matrix

# Import your preferred model here
from model_library import preferred_model

# Debug parameter
DEBUG = False

if DEBUG:
    print("Debug mode is ON. Detailed output will be printed.")

# Load Database
csv_hour_file = '../data/ta_metrics/final_price_ta_metrics.csv'
df = pd.read_csv(csv_hour_file, parse_dates=['Datetime'])

# Define the Sliding Windows for this run
sliding_window = 10 # of days to train on (matrix rows)
price_feature_window = 3  # Window of the number of previous days as features (matrix columns)

# The following +1 is used to include the next row as the test set for model validation
training_sliding_window = sliding_window + 1

# Calculate number of sliding window models to train in the dataset
num_sliding_windows = len(df) - training_sliding_window
if DEBUG:
    print(f"Number of rows in the dataset: {len(df)}")

print(f"Number of sliding windows to train: {num_sliding_windows}")

# Initialize lists to store predictions, actuals, and timestamps
predictions_list = []
actuals_list = []
timestamps_list = []

# Initialize the model
model = preferred_model

for i in range(num_sliding_windows):
    if DEBUG:
        print(f"Processing sliding window {i + 1}/{num_sliding_windows}...")

    # Ensure we do not go out of bounds
```

```
    if i + training_sliding_window >= len(df):
        break

    sliding_window_set = df.iloc[i : i + training_sliding_window]

    # Create feature matrix and target variable for training
    X_train, y_train = create_expanded_feature_matrix(
        sliding_window_set, price_feature_window)

    # Split into training matrix and prediction feature row
    X_train_fit = X_train.iloc[:-1]
    y_train_fit = y_train.iloc[:-1]

    X_to_predict = X_train.iloc[-1:]
    y_to_predict = y_train.iloc[-1]

    # Train the model with the feature matrix
    model.fit(X_train_fit, y_train_fit)

    # Make prediction with the next row of features
    y_predicted = model.predict(X_to_predict)

    # Add bounds checking to catch extreme predictions
    if y_predicted[0] < 0: # Avoid negative price predictions, set
        lower limit to 0
        y_predicted[0] = 0
    if y_predicted[0] > 1000: # limit to 1000, avoiding unrealistic
        predictions
        y_predicted[0] = 1000

    # Store results for this sliding window
    predictions_list.append(y_predicted[0])
    actuals_list.append(y_to_predict)
    if 'Datetime' in sliding_window_set.columns:
        timestamps_list.append(sliding_window_set.iloc[-1]['Datetime
            '])
    else:
        timestamps_list.append(i + training_sliding_window - 1)

# Create prediction vs actual DataFrame
prediction_df = pd.DataFrame({
    'Timestamp': timestamps_list,
    'Predicted': predictions_list,
    'Actual': actuals_list
})
```

4. EXPERIMENTATION

This chapter is dedicated to reviewing all of the relevant information pertaining the experimentation phase of the project. We will review the specifics of the data that was dealt with, and explain the decisions taken for the preparation of such data, and the training of the different machine learning models employed. We will review all of the results of the different models, and discuss in depth the different set-up variations for the models.

4.1. Data Description

The data that was used for this project was exclusively publicly available information, published by the OMIE (*Operador del Mercado Ibérico de Energía*), the operator of the Iberian energy market. The data that was retrieved from their website [10] for this project takes the following shape:

```
MARGINALPDBC;
2018;01;01;1;28.1;6.74;
2018;01;01;2;33;4.74;
2018;01;01;3;32.9;3.66;
2018;01;01;4;28.1;2.3;
2018;01;01;5;27.6;2.3;
2018;01;01;6;24.6;2.06;
2018;01;01;7;20.1;2.06;
2018;01;01;8;19.9;2.06;
2018;01;01;9;19.84;2.3;
2018;01;01;10;19.9;2.3;
2018;01;01;11;19.9;2.3;
2018;01;01;12;19.9;2.3;
2018;01;01;13;23.6;2.3;
2018;01;01;14;25.1;2.3;
2018;01;01;15;23.6;5;
2018;01;01;16;24.6;5;
2018;01;01;17;25.1;5;
2018;01;01;18;27.6;8.85;
2018;01;01;19;27.6;15.93;
2018;01;01;20;28.1;22.02;
2018;01;01;21;32.9;20;
2018;01;01;22;28.1;21.95;
2018;01;01;23;28.1;23.52;
2018;01;01;24;27.6;16.35;
*
```

We can understand and interpret this format following the OMIE's guide: *Modelo de Ficheros para la distribución pública de Información del mercado de electricidad 1.35* [13]. In page number 67, chapter 6.18 we may find the following information regarding the encoding format for the information:

```
6.18 Precios marginales del mercado diario (MARGINALPDBC)

Fichero con los precios marginales del Mercado Diario para cada una de
las horas.
Nombre del fichero: marginalpdbc_aaaammdd.v donde aaaammdd
corresponde a la fecha de sesión y v es la versión del fichero.

Descripción de los campos:
CAMPO DESCRIPCIÓN VALORES VÁLIDOS
Año Año I4 - 20XX
Mes Mes I2 - 1 a 12
Día Día I2 - 1 a 31
Hora Hora I2 - 1 a 25
MarginalPT Precio marginal zona Portuguesa F8.2 - -99999.99 a 99999.99
MarginalES Precio marginal zona Española F8.2 - -99999.99 a 99999.99
```

From this data description we can obtain the following column names: *Año, Mes, Día, Hora, MarginalPT* and *MarginalES*. As a proof of concept, in this investigation we will be focusing on a single time slot, simplifying the data, from multiple data points per day, to a single one. We are exclusively interested in the value of the last column, *MarginalES*, and specifically the row pertaining the 14^{th} hour of each day, which we will refer to as 14H.

After the data is ingested to the **Data analysis:** As previously commented: the in the No estacionario.

No son datos IID - Independientes e identicamente distribuidos.

Hay correlacion en mis datos entre cada dia? Si deberia ser relativamente alta, estacional, pero progresiva y lenta.

For the data visualization portion, I employed various methods, bot plotting values with matplotlib, seaborn and using the Microsoft Data Wrangler Visual Studio Code extension.

Explain some more about the data distribution.

Explain the ups and downs of the data (war and relation to natural gas prices).

Explain the zero price.

4.2. Model Set Up Explanation

How do I use in practice the previously explained system - what parameters did I modify to obtain the final results. Feature engineering, Alpha variation testing in Lasso, Tree depth and number of leafs.

Que modelos y que configs

Cross validation?? Not really done, in another way w the sliding window

cambio parametros pero no la arquitectura del modelo

- Linear Regressor: A linear combination of every feature

No optimization to be done here, just testing different sliding windows

- Lasso Regressor: A linear model that penalizes unused features

Experimenting with alpha using a loop or GridSearchCV to find the optimal value.

As mentioned earlier, tuning the hyperparameter *alpha* enables To optimize our predictions, we will later experiment across a variety of *alpha* hyperparameter values.

To achieve this, all of the parameter values selected will be tested in every sliding window iteration, noting down the *alpha* value that renders the result with the minimum error to the actual data point.

- Random Forest: Soft on peaks - tune hyperparameters like n estimators, max depth, etc.

There are two main parameters that we will focus on tuning. The number of trees in the forest (`n_estimators`) and the maximum depth of the tree branches (`max_depth`) will be tuned.

Similar to the training process of the Lasso Regressor, for every sliding window iteration each combination of parameters will be tested. Likewise, we will save the combination of parameters with the best result, with the least error to the actual data point.

4.3. Results of the Testing

Results as graph/ tables - This would be to compare a model across various iterations. Model best-run comparisons, etc.

Review error rates and compare with other iterations/ batches and other models.

Error medio en todas las predicciones

Y percentiles (expectation shortfall tambien?)

4.4. Discussion

This would be the final explanation that I would give to a colleague of mine, with full details on how I have done everything

5. REGULATORY FRAMEWORK

Not essential for this investigative project because we do not go to market.

5.1. Data Availability

Habria que hablar de las normas, pq si alguien lo usa es para ir a mercado

Current Spanish and European regulations allow for the use of publicly available institutional data for use in educational investigative work.

I need to talk about laws, what data is permissible to use and how. This would be essential for a project that does indeed go to market.

For us, its not that relevant.

5.2. Software & Licenses

For the development of this project I have used a variety of open and closed source utilies and code.

For the sake of organization, I will categorize this section into two distinct parts, required software to reproduce the project, and optional software for ease of production, plannification, organization or otherwise related.

Required:

Python - This was the programming language in wich the entirety of the project was developed [14]

Scikit-learn - library for ml [15]

Pandas - data management [16]

ta library - Technical Analysis library[12]

Light use of Seaborn - data visualization [17]

NumPy - number manipulation [18]

macOS / Windows - closed source platforms across where the project was developed [19] & [20]

Optional:

Git - a distributed version control system [21]

GitHub - Repository hosting platform [22]

Visual Studio Code - open source text editor for my general coding needs [23]

Visual Studio Code extensions such as Microsoft Data Wrangler, Python packages or such

Python Virtual Environment (venv) - This was created as it is best practice to create new Python virtual environments for each new project, as specially in a production environment these usually require certain specific versions that must be met in order for all components to work as intended. An alternative method of dealing with this would be using Conda, and creating with it a new Conda environment, [8]

Overleaf - closed source platform used to compile LaTeX code [24]

LaTeX - open source language used to create the final project document [25]

OpenAI ChatGPT, Google Gemini & Anthropic Claude - closed source large language models used to aid in troubleshooting general coding issues

6. SOCIO-ECONOMIC ENVIRONMENT

Into to this?

6.1. Impact

Why is this relevant?

- Energy Prediction

- Shutdowns

- Price sensitivity

In this project we have

6.2. Project Plan

TODO: Gantt diagram representing time spent in each phase - Design and innovation time frames

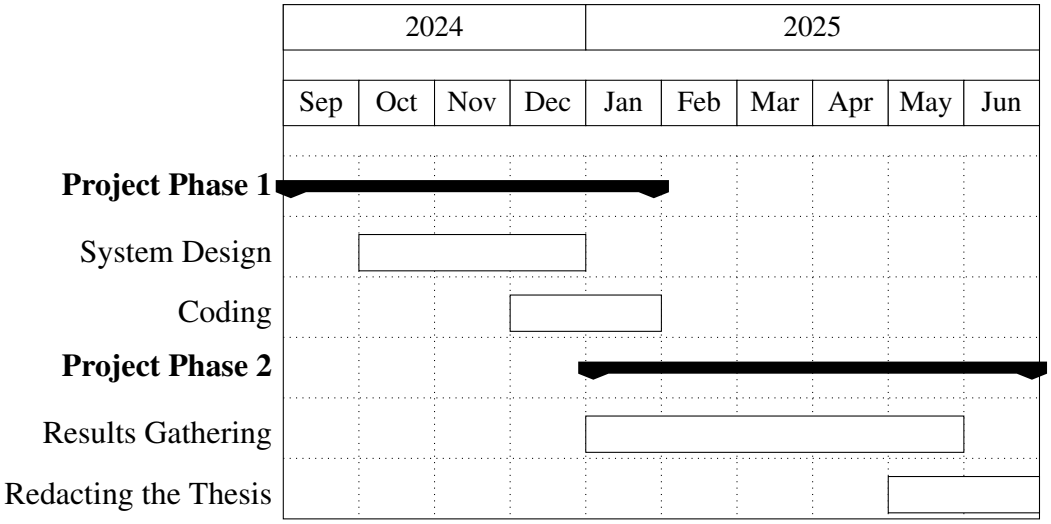

Fig. 6.1. Gantt Diagram of the Project Lifecycle

6.3. Budget

This section will consist of a complete price breakdown of the investigation, consisting of material costs such as the equipment utilized, and the human capital that composed the investigation team. It is notable to highlight that as previously mentioned, no paid license of any sort was obtained for the realization of this project. All of the programs, tools or code, were either *Open Source* or free to use software under exclusive *non-commercial use* licenses.

The most basic requirement for the physical needs of this project is a computer with an desktop operating system. Since the project was written in Python [14], it would be reasonable to consider it platform agnostic. Any computer capable of running any recent operating system, configured with a recent version of Python would be able to run this project. This includes any of the three most popular desktop operating systems, Windows, macOS and any Linux distribution.

The project was mainly tested using Python 3.13 [7] which does require a recent operating system such as macOS 13 or Windows 10. But as for the project itself, there is no platform, processor architecture, or processing power requirements. In my case, I mainly developed the code in my personal laptop, a 2021 MacBook Pro configured with the ARM M1 Pro chip and 16GB of RAM, running the latest available macOS version at this time, macOS 15 [19]. This laptop was obtained from a second-hand store circa June 2024, mainly for personal use, but also helping keep the theoretical project costs down.

The following table outlines the specific equipment costs incurred for the development of this investigative project:

Equipment	**Price (€)**	**Amortization per year (€)**	**Useful Life (years)**	**Usage Time (months)**	**Cost (€)**
MacBook Pro M1 Pro	1250	312.5	4	9.5	247.4

TABLE 6.1. EQUIPMENT AMORTIZATION

Regarding the human capital that was required for the successful development of this project, it will consist exclusively on two people. The director of my bachelor's thesis, professor Emilio Parrado Hernández, PhD, as the expert Machine Learning consultant, and myself as a junior software engineer.

The following table displays the estimated hourly rates of each person, the hours dedicated to the project, and the total cost:

Name	**Role**	**Price (€/hour)**	**Hours**	**Cost (€)**
Emilio Parrado Hernández	Expert ML Consultant (PhD)	200	-	-
Rodrigo De Lama Fernández	Junior Engineer (Pre-Graduate)	30	-	-

TABLE 6.2. HUMAN COSTS

For our final budgeting considerations, other miscellaneous expenses, such as transportation costs to the university for on site meetings with my bachelor's thesis director. Summing up all costs, the following table estimates what this project would have cost to investigate:

Concept	**Cost (€)**
Direct/Material Costs	247.4
Engineering Costs	-
University Carlos III Costs - Expert ML Consultant	-
Total	**-**

TABLE 6.3. FINAL COST BREAKDOWN

7. CONCLUSIONS

In this investigation we have analyzed in depth an innovative methodology for predictive modeling of energy prices with Machine Learning algorithms, enhanced by the usage of technical analysis indicators in feature engineering, in order to enhance prediction precision. This final chapter will review the general conclusions attained throughout the development of this project's Machine Learning based solution for energy price predictions. It will also discuss potential development paths for future work pertaining the system designed in this investigative project

7.1. Revisiting the Objectives of the Project

After finishing the whole project, read it, and reintroduce the objectives to the reader. Remind them of the completion of them

Predicting the prices in an accurate manner using Machine Learning models using feature engineering with technical analysis indicators

7.2. Future work

- Less absolute error

- Menos piradas de modelos

- Better overall precision: better percentile accuracy

- Combinacion de modelos para una mejor prediccion? - La consistencia de Random Forest con los picos de los modelos lineales?

-

BIBLIOGRAPHY

[1] EDEM, *Cómo se fija el precio de la electricidad en españa*, https://edem.eu/como-se-fija-el-precio-de-la-electricidad-en-espana-para-dummies/, Accessed on June 9, 2025, Jun. 2025.

[2] The scikit-learn Development Team, *Scikit-learn: Linear regression*, https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html.

[3] The scikit-learn Development Team, *Scikit-learn: Lasso regression*, https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Lasso.html.

[4] The scikit-learn Development Team, *Scikit-learn: Random forest regressor*, https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html.

[5] Amazon Web Services, *What is hyperparameter tuning?* https://aws.amazon.com/what-is/hyperparameter-tuning/.

[6] K. Montevirgen. "Technical analysis: Is there predictive power in chart data?" (Sep. 16, 2022), [Online]. Available: https://www.britannica.com/money/what-is-technical-analysis.

[7] Python Software Foundation, *Python 3.13*, https://www.python.org/downloads/release/python-3130/, 2025.

[8] Python Software Foundation, *Venv — creation of virtual environments*, https://docs.python.org/3/library/venv.html, 2025.

[9] Anaconda Inc., *Miniconda*, https://www.anaconda.com/docs/getting-started/miniconda/main, 2025.

[10] OMIE, *Precios horarios del mercado diario en españa*, https://www.omie.es/es/file-access-list?parents=/Mercado%20Diario/1.%20Precios&dir=Precios%20horarios%20del%20mercado%20diario%20en%20EspaÃśa&realdir=marginalpdbc, Accessed on June 9, 2025, Jun. 2025.

[11] The Pandas Development Team, *Pandas dataframe*, https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html, 2025.

[12] bukosabino, *Technical analysis library in python (ta)*, https://github.com/bukosabino/ta, 2025.

[13] OMIE, *Modelo de ficheros para la distribución pública de información del mercado de electricidad 1.35*, https://www.omie.es/sites/default/files/2024-06/Formato_ficheros_inf_pub_135_0.pdf, Accessed on June 9, 2025, Jun. 2024.

[14] Python Software Foundation, *Python programming language*, https://www.python.org/about/, 2025.

[15] The scikit-learn Development Team, *Scikit-learn: Machine learning in python*, https://scikit-learn.org/stable/, 2025.

[16] The Pandas Development Team, *Pandas: Python data analysis library*, https://pandas.pydata.org, 2025.

[17] Michael Waskom et al., *Seaborn: Statistical data visualization*, https://seaborn.pydata.org, Used lightly for visualization, 2025.

[18] The NumPy Development Team, *Numpy: Array programming for scientific computing*, https://numpy.org, 2025.

[19] Apple Inc., *Macos sequoia*, https://www.apple.com/macos/macos-sequoia/, 2025.

[20] Microsoft Corpotation, *Windows 11*, https://www.microsoft.com/windows/windows-11, 2025.

[21] Git, *Git: A distributed version control system*, https://git-scm.com, 2025.

[22] GitHub, *Github: Repository hosting platform*, https://github.com/about, 2025.

[23] Microsoft Corporation, *Visual studio code*, https://code.visualstudio.com/docs/editor/whyvscode, Open source code editor, 2025.

[24] Overleaf, *Overleaf: Online LaTeX editor*, https://www.overleaf.com/about, 2025.

[25] LaTeX Project, *Latex: A document preparation system*, https://www.latex-project.org, 2025.